

The Power of Sorted Data

Halving the Problem -- Binary Search

Lecture 25

Data Structures and Algorithms

Where We Left Off

Last lecture we studied **Quicksort** -- the fastest comparison sort in practice.

Property	Quicksort
Best case	$O(n \log n)$
Average case	$O(n \log n)$
Worst case	$O(n^2)$
Space	$O(\log n)$
Stable?	No
In-place?	Yes

With Quicksort and Merge Sort, we now have practical $O(n \log n)$ sorting algorithms.

Today's question: Now that we can sort data efficiently, how do we **search** it?

Part 1: Introduction to Searching

The Baseline -- Linear Search

The Searching Problem

Given a collection of n elements and a target **key**, determine whether the key exists in the collection -- and if so, **where**.

Searching is one of the most frequent operations in computing:

- **Databases:** Finding a customer record by ID
- **File systems:** Locating a file by name
- **Autocomplete:** Finding matching words as you type
- **Spell-checkers:** Verifying whether a word exists in a dictionary
- **Contact lists:** Finding a phone number by name

Connection to sorting: "This is why we spent four lectures learning to sort. Sorting is not the end goal -- it enables **faster searching**."

Linear Search -- The Baseline

The simplest approach: examine every element from start to end until the target is found.

```
LinearSearch(A, n, key):  
    for i = 0 to n-1:  
        if A[i] == key:  
            return i        // Found at index i  
    return -1               // Not found
```

Properties:

- Works on **unsorted** and **sorted** arrays -- no preconditions
- **Best case:** $O(1)$ -- target is the first element
- **Average case:** $O(n)$ -- target is roughly in the middle
- **Worst case:** $O(n)$ -- target is at the end, or not present at all
- **Space:** $O(1)$

Can we do better? **Yes -- if the data is sorted.**

The Intuition: Why Sorted Data Helps

Imagine looking up the word "quicksort" in a physical dictionary.

Strategy 1 (Linear Search): Start at page 1, read every word on every page until you find "quicksort." This works but could take thousands of pages.

Strategy 2 (What you actually do): Open the dictionary roughly in the middle. You see words starting with "M." Since "Q" comes after "M," you ignore the entire first half and open the middle of the second half. You see "R." Since "Q" comes before "R," you ignore everything after "R." Each time, you **eliminate half** the remaining pages. This is **Binary Search**. With each comparison, you cut the search space in half. A dictionary with 100,000 words needs at most $\lceil \log_2(100,000) \rceil = 17$ lookups.

Part 2: Binary Search

The Algorithm

The Binary Search Idea

Precondition: The array must be **sorted** in non-decreasing order.

Algorithm:

1. Compare the target **key** with the **middle** element of the current search range
2. If they are equal -- **found!**
3. If **key < middle** -- the target must be in the **left half** (discard the right half)
4. If **key > middle** -- the target must be in the **right half** (discard the left half)
5. Repeat until the target is found or the search range is empty

Why it works: Since the array is sorted, every element to the left of the middle is smaller, and every element to the right is larger. One comparison eliminates exactly half the remaining candidates.

This is **Divide and Conquer** applied to searching -- the same paradigm behind Merge Sort and Quicksort.

Recursive Binary Search

```
BinarySearch(A, low, high, key):  
    if low > high:  
        return -1                // Base case: not found  
    mid = low + (high - low) / 2  // Safe midpoint calculation  
    if A[mid] == key:  
        return mid                // Found!  
    else if key < A[mid]:  
        return BinarySearch(A, low, mid - 1, key)    // Search left  
    else:  
        return BinarySearch(A, mid + 1, high, key)    // Search right
```

Reading this algorithm:

- **Base case:** `low > high` means the search range is empty -- the key is not in the array
- **Recursive case:** Compare with the middle, then recurse on the appropriate half
- The pivot element `A[mid]` is excluded from both recursive calls -- we already checked it

Important: We use `low + (high - low) / 2` instead of `(low + high) / 2`. Why? Because `low + high` can exceed `INT_MAX` and cause **integer overflow**. This subtle bug went undetected in Java's standard library for nearly a decade before being discovered in 2006.

Traced Example 1: Successful Search

Array: **[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]** -- Search for key = **23**

```
Call 1: low=0, high=9  
mid = 0 + (9-0)/2 = 4  
A[4] = 16  
23 > 16 --> search right half A[5..9]
```

```
Call 2: low=5, high=9  
mid = 5 + (9-5)/2 = 7  
A[7] = 56  
23 < 56 --> search left half A[5..6]
```

```
Call 3: low=5, high=6  
mid = 5 + (6-5)/2 = 5  
A[5] = 23  
23 == 23 --> FOUND! Return index 5.
```

Total comparisons: 3. Array size: 10. $\lceil \log_2(10) \rceil = 4$. We found it in fewer than the maximum because we got lucky with the midpoint.

Traced Example 2: Unsuccessful Search

Same array: **[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]** -- Search for key = **25**

```
Call 1: low=0, high=9  
mid=4, A[4]=16. 25 > 16 --> search A[5..9]
```

```
Call 2: low=5, high=9  
mid=7, A[7]=56. 25 < 56 --> search A[5..6]
```

```
Call 3: low=5, high=6  
mid=5, A[5]=23. 25 > 23 --> search A[6..6]
```

```
Call 4: low=6, high=6  
mid=6, A[6]=38. 25 < 38 --> search A[6..5]
```

```
Call 5: low=6, high=5  
low > high --> NOT FOUND. Return -1.
```

Total comparisons: 4. The search exhausted all possibilities. Each call halved the search space: 10 elements, then 5, then 2, then 1, then 0.

How the Search Space Shrinks

Let us trace the **size of the search range** at each step for the unsuccessful search:

```
Call 1: 10 elements [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
Call 2:  5 elements [23, 38, 56, 72, 91]
Call 3:  2 elements [23, 38]
Call 4:  1 element  [38]
Call 5:  0 elements [] --> STOP
```

The pattern: $10 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 0$. Each step reduces the problem by roughly half.

After k comparisons, the search space is $n/2^k$. We stop when $n/2^k < 1$, which gives us $k = \lceil \log_2 n \rceil$.

For $n = 1,000,000$: at most $\lceil \log_2(1,000,000) \rceil = 20$ comparisons. Linear search might need all 1,000,000.

Part 3: Iterative Binary Search

Eliminating the Recursion

Iterative Binary Search

The same logic, but using a loop instead of recursion:

```
BinarySearchIterative(A, n, key):  
    low = 0  
    high = n - 1  
    while low <= high:  
        mid = low + (high - low) / 2  
        if A[mid] == key:  
            return mid          // Found!  
        else if key < A[mid]:  
            high = mid - 1      // Discard right half  
        else:  
            low = mid + 1       // Discard left half  
    return -1                  // Not found
```

Why use the iterative version?

- **Space:** $O(1)$ -- no recursion stack. The recursive version uses $O(\log n)$ stack space.
- **Performance:** Slightly faster in practice -- no function call overhead per step
- **Same time complexity:** $O(\log n)$ in both versions

The iterative version is generally preferred in production code.

Iterative Trace: Searching for key = 23

Array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

```
Iteration 1: low=0, high=9  
mid=4, A[4]=16. 23 > 16 --> low = 5  
  
Iteration 2: low=5, high=9  
mid=7, A[7]=56. 23 < 56 --> high = 6  
  
Iteration 3: low=5, high=6  
mid=5, A[5]=23. 23 == 23 --> FOUND! Return 5.
```

Same result as the recursive version, same number of comparisons. The only difference is the mechanism: a `while` loop replaces the recursive calls, and `low` and `high` are updated in place instead of being passed as arguments.

Part 4: Binary Search Analysis

Why $O(\log n)$ Changes Everything

Recurrence Relation

The recursive binary search satisfies:

$$T(n) = T(n/2) + O(1)$$

- **One subproblem** of **half the size** -- we recurse on either the left or right half, never both
- **$O(1)$ work** per level -- one comparison and one midpoint calculation

Solving by expansion:

$$T(n) = T(n/2) + c = T(n/4) + 2c = T(n/8) + 3c = \dots = T(1) + c \cdot \log_2 n$$

After k halvings, we reach $T(1)$ when $n/2^k = 1$, giving $k = \log_2 n$.

$$T(n) = O(\log n)$$

By the **Master Theorem** (Case 2 with $a = 1$, $b = 2$, $f(n) = O(1)$): this also gives $T(n) = O(\log n)$.

Complexity Summary

Property	Recursive	Iterative
Best case	$O(1)$	$O(1)$
Average case	$O(\log n)$	$O(\log n)$
Worst case	$O(\log n)$	$O(\log n)$
Space	$O(\log n)$ (stack)	$O(1)$

Best case: The target happens to be exactly at the midpoint on the very first comparison.

Worst case: The target is not present, or it is found at the very last level of recursion -- we perform $\lceil \log_2 n \rceil$ comparisons.

Binary Search vs Linear Search

Feature	Linear Search	Binary Search
Time (worst)	$O(n)$	$O(\log n)$
Time (best)	$O(1)$	$O(1)$
Requires sorted?	No	Yes
Space (iterative)	$O(1)$	$O(1)$
Space (recursive)	N/A	$O(\log n)$

The Scale Demonstration

Array size (n)	Linear Search (worst)	Binary Search (worst)
1,000	1,000 comparisons	10 comparisons
1,000,000	1,000,000 comparisons	20 comparisons
1,000,000,000	1 billion comparisons	30 comparisons

Binary search is exponentially faster. The cost of sorting first ($O(n \log n)$) is repaid after just a few searches.

Part 5: Common Mistakes and Edge Cases

Getting Binary Search Right

The Four Classic Pitfalls

From Thareja: *"Binary search is one of the most misimplemented algorithms -- studies show only about 10% of programmers write a correct binary search on their first attempt."*

Pitfall 1: Off-by-One in the Loop Condition

```
WRONG: while low < high      // Misses the case when low == high
RIGHT: while low <= high    // Checks the last remaining element
```

If `low == high`, there is exactly one element left to check. Using `<` instead of `<=` skips it entirely, causing false negatives.

Pitfall 2: Integer Overflow in Midpoint Calculation

```
WRONG: mid = (low + high) / 2    // low + high can overflow INT_MAX
RIGHT: mid = low + (high - low) / 2 // Always safe
```

For arrays with more than $\sim 10^9$ elements (or when indices are 32-bit), `low + high` can exceed $2^{31} - 1$.

The Four Classic Pitfalls (continued)

Pitfall 3: Not Updating low/high Correctly

```
WRONG: high = mid      // mid was already checked! Infinite loop risk.  
RIGHT: high = mid - 1  // Exclude mid from the next search range.  
  
WRONG: low = mid       // Same problem.  
RIGHT: low = mid + 1
```

If you set `high = mid` or `low = mid`, the element at `mid` is re-examined in the next iteration. When `low == high == mid`, the loop never terminates.

Pitfall 4: Forgetting the Precondition

Binary search **only works on sorted arrays**. If the array is unsorted, the result is unpredictable -- it might return a wrong index, miss an element that exists, or even appear to find an element that is not at the returned index. Always verify the array is sorted before searching.

Part 6: Applications and Variants

Beyond Simple Search

Real-World Applications of Binary Search

Finding elements in sorted databases and lookup tables

Every database index that supports range queries uses binary search (or its tree-based generalization, B-Trees -- which we studied earlier).

Maintaining sorted order

Use binary search to find the correct insertion position for a new element in a sorted array: $O(\log n)$ to find the position, then $O(n)$ to shift elements.

Computing integer square roots

To find $\lfloor \sqrt{n} \rfloor$: binary search on $[1, n]$ for the largest x where $x^2 \leq n$.

Git bisect

Finding the commit that introduced a bug. Mark one commit as "good" and one as "bad." Git uses binary search on the commit history to find the exact commit where the bug was introduced -- testing $O(\log n)$ commits instead of all n .

Variants of Binary Search

Standard binary search finds **any** occurrence of the key. But what if there are duplicates?

Find First Occurrence

```
FindFirst(A, n, key):
    low = 0, high = n - 1, result = -1
    while low <= high:
        mid = low + (high - low) / 2
        if A[mid] == key:
            result = mid          // Record this, but keep searching left
            high = mid - 1        // There might be an earlier occurrence
        else if key < A[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return result
```

Key change: When we find the key, we do not stop immediately. Instead, we record the index and continue searching the **left half** for a possibly earlier occurrence. This still runs in $O(\log n)$.

Variants of Binary Search (continued)

Find Last Occurrence

```
FindLast(A, n, key):  
    low = 0, high = n - 1, result = -1  
    while low <= high:  
        mid = low + (high - low) / 2  
        if A[mid] == key:  
            result = mid          // Record this, but keep searching right  
            low = mid + 1         // There might be a later occurrence  
        else if key < A[mid]:  
            high = mid - 1  
        else:  
            low = mid + 1  
    return result
```

Other Variants

- **Floor:** Find the largest element $\leq$ key
- **Ceiling:** Find the smallest element $\geq$ key
- **Count occurrences:** `FindLast(key) - FindFirst(key) + 1`

These variants appear frequently in coding interviews and competitive programming.

Part 7: Summary and What is Next

Today's Key Concepts

The Searching Problem:

- Given n elements and a target key, find the key (or determine it is absent)
- Linear search works on any array in $O(n)$
- Binary search works on **sorted** arrays in $O(\log n)$

Binary Search:

- Compare with the middle element, discard half the search space
- Recursive version: $O(\log n)$ time, $O(\log n)$ space
- Iterative version: $O(\log n)$ time, $O(1)$ space
- Precondition: the array **must be sorted**

Common Pitfalls:

- Off-by-one: use `<=` not `<`
- Overflow: use `low + (high - low) / 2`
- Stale mid: update to `mid - 1` or `mid + 1`, never just `mid`

The Updated Complexity Table

Algorithm	Best	Average	Worst	Space	Stable?	In-Place?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes

Searching Algorithms:

Algorithm	Best	Average	Worst	Space	Requires Sorted?
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	No
Binary Search (iterative)	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Yes
Binary Search (recursive)	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Yes

Review Questions

1. Trace the iterative binary search on `[3, 7, 11, 15, 19, 23, 27, 31]` for key = 19 and key = 20. Show `low`, `high`, and `mid` at each step. How many comparisons does each search require?
2. An array has 1 billion elements. What is the maximum number of comparisons binary search needs? What about linear search?
3. A student writes `mid = (low + high) / 2` and gets incorrect results when searching a very large array. Explain what goes wrong and how to fix it.
4. Given the sorted array `[1, 3, 3, 3, 5, 7, 9]`, modify standard binary search to find the **first** occurrence of 3 and the **last** occurrence of 3. What are the indices?
5. Your data changes frequently -- items are inserted and deleted thousands of times per second. Is it practical to keep the data in a sorted array and use binary search? What alternative data structure would you use instead?

What Comes Next

Lecture 26 brings the **Comprehensive Analysis and Review** -- every sorting and searching algorithm side by side.

We will:

- Build the complete comparison table with all algorithms from this unit
- Develop a **decision framework** for choosing the right algorithm
- Study what real-world languages actually use: Timsort (Python), Dual-Pivot Quicksort (Java), Introsort (C++ STL)
- Work through practice problems tracing and comparing algorithms

After six lectures, you will have a complete understanding of why, when, and how to sort and search data efficiently.

